

BAŞLIK: Q-LEARNING: DEĞER TABANLI PEKİŞTİRMELİ ÖĞRENME

TÜR: Akademik/Teknik Detaylı Anlatım

BÖLÜM 1: Q-DEĞERİ (Q-VALUE) NEDİR?

Bir önceki dosyada (21.1.1) ajanın "beyni" olan politika (π) kavramını tanımlamıştık. Q-Learning, bu politikayı dolaylı yoldan, bir ara değer fonksiyonu üzerinden öğrenen, değer tabanlı (value-based) bir algoritmadır. Bu ara değere "Q-değeri" denir.

Tanım: $Q(s, a)$ fonksiyonu, "s durumundayken a eylemini yapar ve ondan sonra en akıllıca şekilde davranmaya devam edersem, epizot sonuna kadar beklediğim toplam indirgenmiş ödül ne kadardır?" sorusunun cevabıdır. Q harfi İngilizce "Quality" (kalite) kelimesinden gelir — bir eylemin, belirli bir durumdaki "kalitesini" ölçer.

Bunu bir şehir haritasında rota bulma problemine benzetelim: Bulunduğunuz her kavşak (durum), ve o kavşaktan çıkan her yol (eylem) için, "bu yolu seçersem hedefe toplamda ne kadar hızlı/verimli varırım" bilgisini önceden bilerseniz, her kavşakta en yüksek puanlı yolu seçerek optimum rotayı bulursunuz. Q-tablosu tam olarak budur: her (durum, eylem) çifti için önceden hesaplanmış bir "gelecek vaadi" puanı.

1.1. Optimum Politikayı Q-Değerinden Türetmek

Eğer bir ajan, her (durum, eylem) çifti için gerçek Q-değerini biliyorsa, artık ayrıca bir politika öğrenmesine gerek kalmaz: herhangi bir durumda yapılması gereken en iyi eylem, basitçe o durumdaki en yüksek Q-değerine sahip eylemdir.

$$\pi^*(s) = \operatorname{argmax}_a Q(s, a)$$

Bu yüzden Q-Learning'in tüm amacı, doğru $Q(s,a)$ değerlerini deneme-yanılma yoluyla, adım adım yaklaşık olarak öğrenmektir. Öğrenmenin başında bu tablo genellikle sıfırlarla (ya da rastgele küçük değerlerle) başlatılır; ajan hiçbir şey bilmiyordur. Ortamı binlerce kez deneyimledikçe tablo, gerçek değerlere doğru yakınsar (converge).

BÖLÜM 2: BELLMAN DENKLEMİ VE GÜNCELLEME KURALI

Q-tablosunu güncellemek için kullanılan formül, Richard Bellman'ın adını taşıyan ve dinamik programlamanın temelini oluşturan bir özyinelemeli (recursive) ilişkiden türetilir. Fikir şudur: "Bir (durum, eylem) çiftinin gerçek değeri, o anda alınan ödül artı, bir sonraki durumdan itibaren elde edilebilecek en iyi değerin indirgenmiş halidir."

$$Q(s,a) \leftarrow Q(s,a) + \alpha \cdot [r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a)]$$

2.1. Formülün Parça Parça Anlamı

- $Q(s,a)$: O anki (eski) tahminimiz — henüz güncellenmemiş değer.
- α (alfa, öğrenme oranı): 0 ile 1 arası bir katsayı; yeni bilgiye ne kadar "kanacağımızı" belirler. $\alpha=1$ ise eski bilgiyi tamamen unutup sadece son deneyime güveniriz; α 'ya yakın 0 ise neredeyse hiç güncelleme

yapmayız.

- r : Az önce a eylemini yaparak gerçekten aldığımız (gözlemlenen) anlık ödül.
- $\gamma \cdot \max_{a'} Q(s',a')$: "Hedef" (target) değerin geleceğe bakan kısmı — yeni vardığımız s' durumunda, elimizdeki bilgiye göre en iyi eylemin Q -değeri, indirgenerek eklenir.
- $[r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a)]$: Buna "TD hatası" (Temporal Difference Error) denir — "tahminimiz ile gerçekte gördüğümüz + tahmin edilen gelecek" arasındaki fark. Gradyan iniş mantığındaki "hata" ile birebir aynı rolü oynar: hata ne kadar büyükse güncelleme o kadar büyük olur.

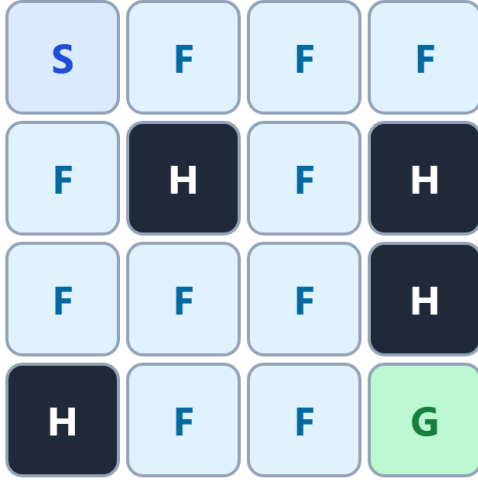
Bu formülün en can alıcı özelliği "bootstrapping" denen tekniktir: $Q(s',a')$ değeri de aslında henüz kesinleşmemiş bir tahmindir, ama yine de onu bir sonraki tahminin parçası olarak kullanırız. Zamanla bu tahminler birbirini düzeltir ve tüm tablo gerçek değerlere yakınsar — tıpkı bir çok değişkenli denklem sisteminin, yinelemeli olarak çözülmesi gibi.

BÖLÜM 3: FROZENLAKE ÖRNEĞİ ÜZERİNDE UYGULAMA

Teoriyi somutlaştırmak için Gymnasium kütüphanesinin klasik "FrozenLake-v1" ortamını kullanacağız (bkz. 21.2.2_q_learning_frozenlake.py). Ortam 4x4'lük bir buz gölüdür:

FrozenLake Ortami ve Ogrenilen Q-Tablosu

Q-Tablosu (ogrenilen deger tahminleri)



Durum	Sol	Asagi	Sag	Yukari
(0,0)	0.12	0.31	0.45	0.09
(0,1)	0.20	0.38	0.22	0.15
(1,0)	0.18	0.52	0.11	0.07
...	...	...	...	...

Şekil 2 — FrozenLake ortamı: S=Başlangıç, F=Donmuş (güvenli) buz, H=Delik (düşerse epizot biter, ödül 0), G=Hedef (ulaşırsa ödül +1). Sağda, eğitim sonunda öğrenilmiş örnek bir Q-tablosu görülüyor.

16 durum (4x4 ızgaradaki her hücre) ve 4 eylem (Sol, Aşağı, Sağ, Yukarı) vardır — bu yüzden Q-tablosu 16x4 = 64 hücrelik küçük bir tablodur ve doğrudan bir NumPy dizisiyle temsil edilebilir (`np.zeros((16, 4))`). Ajan sadece Hedefe (G) ulaştığında +1 ödül alır; her diğer adımda ödül 0'dır. "is_slippery=True" modunda ortam stokastiktir: ajan "sağa git" komutu verse bile buz kaygan olduğu için %33 ihtimalle yukarı ya da aşağı kayabilir — bu da $P(s'|s,a)$ geçiş fonksiyonunun neden olasılıksal olduğunu somut olarak gösterir (bkz. 21.1.1, Bölüm 3).

Ödülün sadece hedefe ulaşıldığında verilmesi, "seyrek ödül" (sparse reward) problemine iyi bir örnektir: ajan, epizotların büyük çoğunluğunda (rastgele dolaşırken bir deliğe düşene ya da adım sınırına ulaşana kadar) hiç ödül almaz. Bu da Bölüm 4'te göreceğimiz keşif stratejisinin (ϵ -greedy) neden bu kadar kritik olduğunu açıklar: ajan yeterince keşif yapmazsa hedefe rastlayıp o pozitif ödülü hiç görmeyebilir.

BÖLÜM 4: HİPERPARAMETRELER VE EĞİTİM STRATEJİSİ

Q-Learning'in pratikte çalışması, birkaç hiperparametrenin doğru ayarlanmasına bağlıdır:

- Öğrenme Oranı (α): FrozenLake gibi küçük, deterministik olmayan ortamlarda genelde 0.1 - 0.8 arası bir değer iyi çalışır.
- İndirim Faktörü (γ): Genelde 0.9 - 0.99 arası seçilir; hedefe ulaşmak birkaç adım gerektirdiği için ajanın "sabırlı" olması (yüksek γ) gerekir.
- Epsilon ve Epsilon Decay: Eğitim $\epsilon=1.0$ (tamamen rastgele) ile başlar ve her epizottan sonra küçük bir çarpınla (örn. 0.995) çarpılarak azaltılır, böylece ajan zamanla daha çok sömürü yapar (bkz. 21.1.1, Bölüm 4.1).

- Epizot Sayısı: Tablonun yakınsaması için binlerce epizot (örn. 10.000+) gerekebilir; küçük ortamlarda bu saniyeler sürer.

4.1. Q-Learning'in Sınırı: Tablo Patlaması

FrozenLake'te durum sayısı sadece 16 olduğu için tam bir tablo tutmak kolaydır. Ama CartPole gibi sürekli (continuous) durum uzayına sahip bir ortamda (konum, hız, açı, açısal hız — 4 gerçek sayı) olası durum sayısı sonsuzdur; bir tabloyu asla tam olarak dolduramayız. Bu "boyutluluk laneti" (curse of dimensionality) problemi, bizi bir sonraki dosyada (21.3.1) ele alacağımız çözüme götürür: Q-tablosunu, sonlu bir tablo yerine bir fonksiyon (örneğin bir yapay sinir ağı) ile yaklaşık olarak temsil etmek.

SONUÇ: UYGULAMA ALANLARI

Tablo tabanlı Q-Learning, durum uzayı küçük ve ayrık olduğunda son derece etkilidir ve şu alanlarda doğrudan kullanılır:

1. Izgara Tabanlı Oyunlar ve Labirent Problemleri: FrozenLake, Taxi-v3 gibi sınırlı durum uzaylı Gymnasium ortamları.
2. Basit Robotik Görevler: Sınırlı sayıda sensör durumuna sahip küçük ölçekli navigasyon problemleri.
3. Envanter ve Kaynak Yönetimi: Stok seviyesi gibi ayrık durumlara sahip basit karar problemleri.
4. Eğitim ve Prototipleme: Q-Learning, çok daha karmaşık algoritmaları (DQN, Policy Gradient) öğrenmeden önce pekiştirmeli öğrenmenin temel mekaniğini kavramak için en yaygın kullanılan başlangıç noktasıdır — bkz. 21.2.2_q_learning_frozenlake.py / .ipynb dosyalarındaki tam kod uygulaması.

Bir sonraki dosyada (21.3.1), durum uzayı büyük ya da sürekli olduğunda Q-tablosunun yerini alan Politika Gradyanı (Policy Gradient) ve Derin Pekiştirmeli Öğrenme (DQN) yaklaşımlarını inceleyeceğiz.